JavaScript String Format: The Best 3 Ways To Do It

https://medium.com/@onlinemsr/javascript-string-format-the-best-3-ways-to-do-it-c6a12b4b94ed

JavaScript String Format

Strings are a primitive data type in JavaScript. The JavaScript string format comes in handy when you want to construct a string dynamically with placeholders that need to be replaced with values at runtime. You may also need to format numbers and money values into strings sometimes. This is where string formatting in JavaScript comes in.

In this post, we'll explore:

- Different ways to format variable string
- Format string using plus (+) sign
- Format using placeholders
- Using string interpolation to format strings
- String format number
- string format money

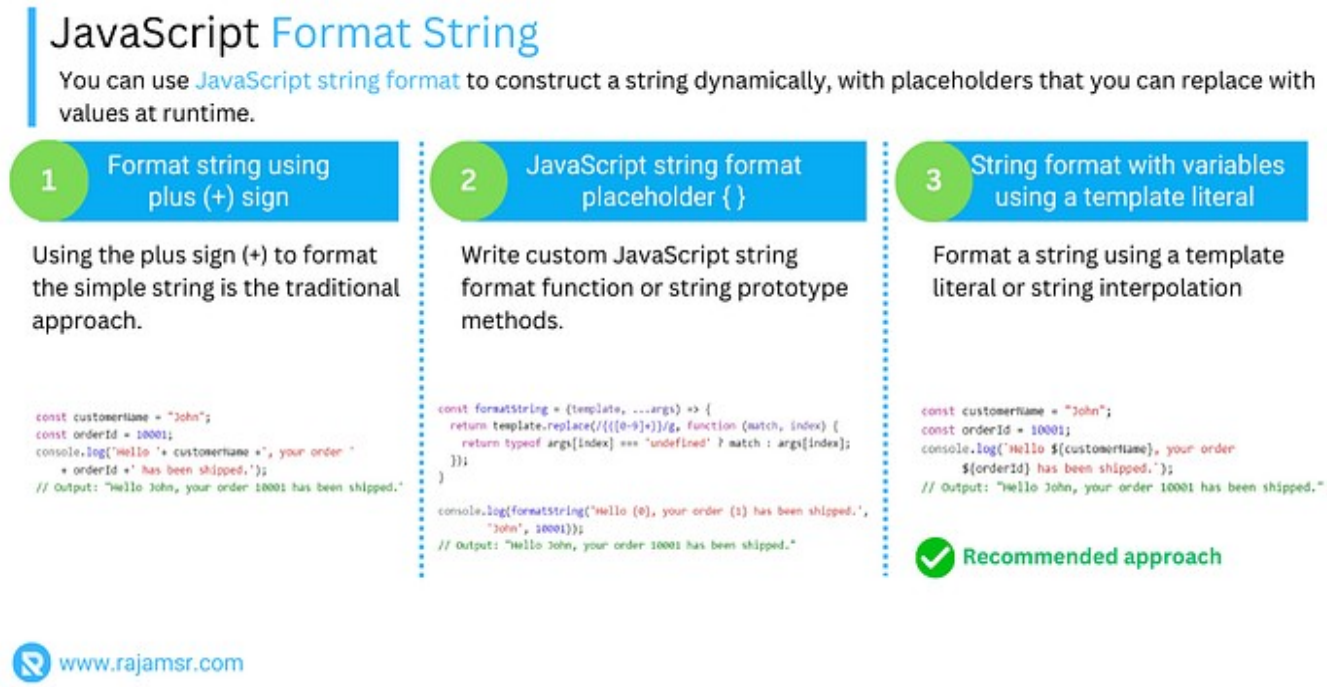
Let's look at each item in detail.

Before diving deep into the javascript string format, if you want to build secure JavaScript and Node applications download the "JavaScript Security Cookbook (Over 40+ recipes)" for FREE!

JavaScript Security Cookbook

JavaScript string format in three different ways

One of the most common reasons to format a string is to print JavaScript variables in a string.
There are many ways to format a JavaScript string.
In this article, we'll look at the top three methods to format strings with variables.



Different types of string formatting in JavaScript

Method 1: Format the JavaScript string using the plus (+) sign

Using the plus sign (+) to format the simple string is the traditional approach. With this method, you have to insert a JavaScript variable in a string separated by the + sign. This approach is similar to the JavaScript string concatenation process.

```
const customerName = "John";
const orderId = 10001;
console.log('Hello '+ customerName +', your order '+ orderId +' has
been shipped.');
// Output: "Hello John, your order 10001 has been shipped."
```

Inserting any arithmetic expression in string formatting without parenthesis will result in unexpected results. The following example prints the unexpected result. The reason for this is that before adding another number, the number is converted to a JavaScript string.

```
console.log('Sum of 5+5 is ' + 5 + 5);
// Output: "Sub of 5+5 is 55."
```

You may simply solve this issue by enclosing integers in parentheses. In this situation, it will first total the integers and then add them to the string.

```
console.log('Sum of 5 + 5 is ' + (5 + 5));
// Output: "Sum of 5 + 5 is 10"
```

Method 2: JavaScript string format placeholder { }

Unlike other programming languages like C#, JavaScript does not support built-in string formatting functions with placeholders { }.
We have to write custom JavaScript string format functions or string prototype methods.

```
const formatString = (template, ...args) => {
  return template.replace(/{([0-9]+)}/g, function (match, index) {
    return typeof args[index] === 'undefined' ? match : args[index];
```

```
  });
}
console.log(formatString('Hello {0}, your order {1} has been
shipped.', 'John', 10001));
// Output: "Hello John, your order 10001 has been shipped."
```

In the above example, the `formatString()` method accepts a template string and args as parameters. Using a `regular expression`, each placeholder is replaced with the corresponding value from args.
You can write this as a JavaScript string `prototype` method, as shown in the below example:

```
String.prototype.format = function () {
  var args = arguments;
  return this.replace(/{([0-9]+)}/g, function (match, index) {
    return typeof args[index] == 'undefined' ? match : args[index];
  });
};
console.log('Hello {0}, your order {1} has been
shipped.'.format('John', 10001));
// Output: "Hello John, your order 10001 has been shipped."
```

Using this approach, you can call the `format()` method on any string with placeholders.
The issue with this approach is when the placeholder count grows. It will be difficult to determine the index position and value to pass as arguments.

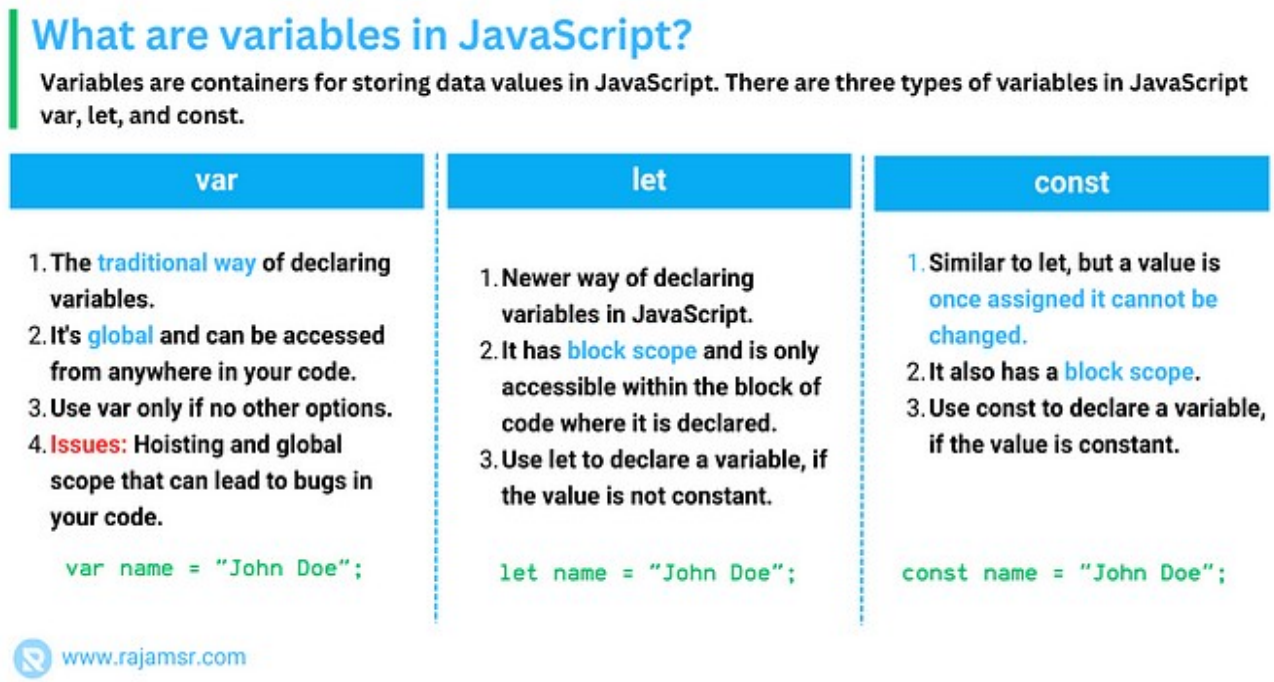
Method 3: String format with variables using a template literal

One of the cleanest and most straightforward methods is to format a string using a template literal or string interpolation. Instead of using a single or double quoted string, we must use the backtick (`) to encapsulate the string in this approach.
JavaScript variables can be placed within parenthesis prefixed with the `$` sign. If this variable is placed in a JavaScript String enclosed with a backtick, then the variable will be expanded with a value during runtime.

```
const customerName = "John";
const orderId = 10001;
```

```
console.log(`Hello ${customerName}, your order ${orderId} has been shipped.`);
// Output: "Hello John, your order 10001 has been shipped."
```

When string format with variables is used in `template literals`, you do not need to worry about variable index position.



JavaScript Variable Types

String format number

Another useful method for formatting strings is to format numbers with commas. You can do this using the built-in `toLocaleString()` method. The `toLocaleString()` string method formats a number according to the user's locale settings

```
const number = 12345.55;
console.log(number.toLocaleString());
// Output: "12,345.55"
```

This code will output "12,345.55" in locales that use the comma as the decimal separator, such as the United States and India. This method is useful when you want to display numbers in a format that's familiar to the user.

For example, if you want to use the `French` locale, you can pass "`fr-FR`" as the locales parameter:

```
const number = 12345.50;
const formattedNumber = number.toLocaleString("fr-FR");
console.log(formattedNumber);
// Output: "12 345,5"
```

String interpolation format number

In JavaScript, you can use string interpolation to format numbers using template literals and the `${ }` syntax.

Here's an example of how you can use string interpolation to format numbers in JavaScript:

```
const number = 10;
const formattedString = `The number is: ${number}`;
console.log(formattedString);
// Output: The number is: 10
```

String format money

To format money as a string in JavaScript, you can use the `toLocaleString()` method. This method converts a number to a string with a language-sensitive representation of the money.

Here's an example:

```
const money = 1234.50;
let options = { style: 'currency', currency: 'USD' }
const formattedMoney = money.toLocaleString('en-US', options);
console.log(formattedMoney);
// Output: "$1,234.50"
```

In this example, the `toLocaleString()` method is called on the money variable with the `'en-US'` locale and options object specifying the style as `'currency'` and the currency as `'USD'`. This formats the money variable as a US dollar currency string. The formatted string is printed on the browser console.

You can replace `'USD'` with the appropriate currency code for your locale.

The `toLocaleString()` method also has other options you can use to customize the formatting, such as specifying the minimum and maximum number of fractional digits.

String format table structure

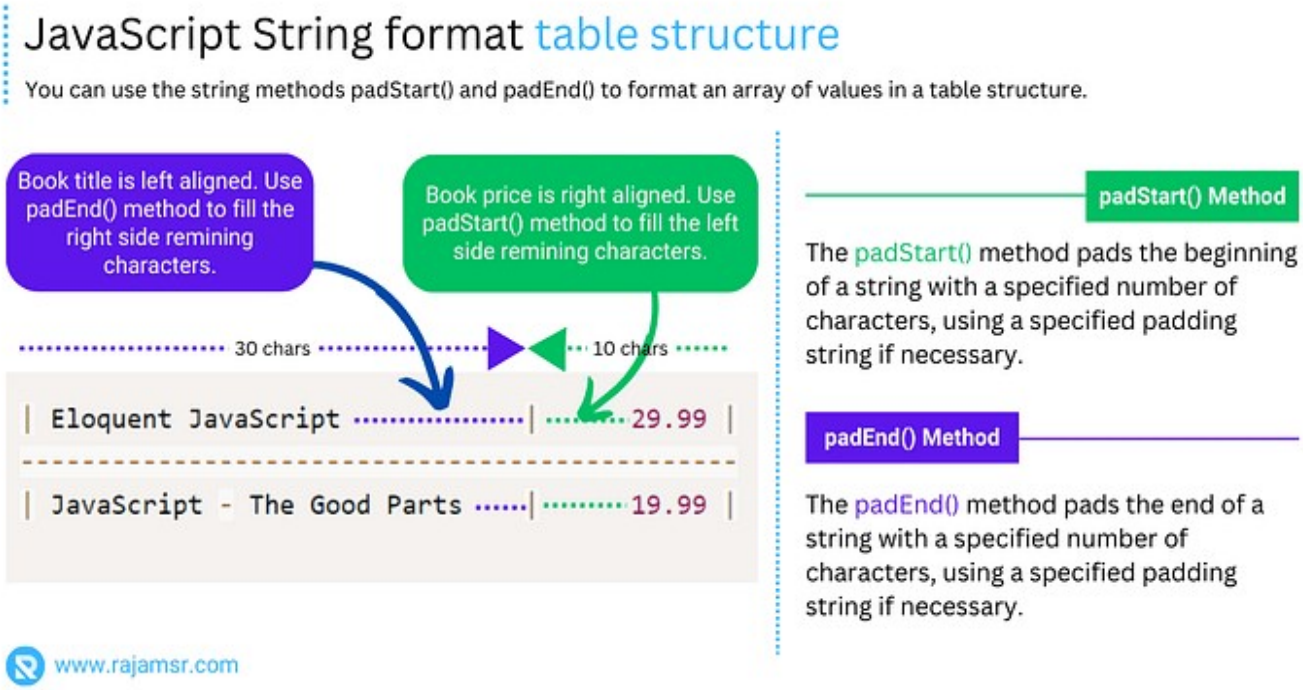
You can use the string methods `padStart()` and `padEnd()` to format an array of values in a table structure. It will pad the current string with a given string (possibly repeated) so that the resulting string reaches a given length. The padding is applied from the start (if the padStart() method is used) or end (if the padEnd() method is used) of the current string.
Let's assume you have an array of book objects and want to print them in a table structure. Here is an example:

```
const books = [
  { title: 'Eloquent JavaScript', price: 29.99 },
  { title: 'JavaScript: The Good Parts', price: 19.99 }
];
const titleColumnWidth = 30;
const priceColumnWidth = 10;
books.forEach(b => {
  let title = b.title.padEnd(titleColumnWidth, ' ');
  let price = b.price.toString().padStart(priceColumnWidth, ' ');

  console.log(`| ${title} | ${price} |`)
  console.log('-'.padStart(titleColumnWidth + priceColumnWidth +
7,'-'))
})
```

The above code will print the book object array in the following format:

```
| Eloquent JavaScript            |      29.99 |
-----------------------------------------------
| JavaScript: The Good Parts     |      19.99 |
```



String format table structure

Conclusion

In this post, we've explored different JavaScript string format approaches.

Using a plus sign is the traditional approach to formatting the string. You can use the custom function or string prototype method to format the string. This approach becomes complex when the number of placeholders grows. Using string interpolation to format the string is a simple and concise way.

Using the toLocaleString() method, you can convert numbers and money into locale-specific strings.

Which approach are you going to use to format the string?